Rapport de Projet Final — N3

Life Goals API

Étudiant : Chebbi Anas

Date de rendu : 15 mars 2026

Repo GitHub : https://github.com/anas-dev523/Life-goals

---

1. Présentation

Sujet choisi

L'application développée est une **API REST de gestion d'objectifs personnels** (Life Goals). Elle permet à un utilisateur de créer, consulter, modifier et supprimer des objectifs de vie. Chaque objectif possède un titre et un statut (par défaut : « En cours »).

Ce sujet a été choisi pour sa simplicité de modélisation et sa pertinence dans le cadre d'un projet pédagogique : le modèle de données est clair, les cas d'usage sont variés et couvrent l'ensemble des opérations CRUD.

Stack technique

Composant	Technologie choisie	Justification
Langage / Framework	Node.js + Express	Vu en cours, syntaxe simple et rapide
Base de données	SQLite via better-sqlite3	Aucun serveur externe requis
Tests	Jest + Supertest	Vu en cours, adapté aux tests d'API HTTP
Linter	ESLint v10	Détecte les erreurs de syntaxe et de style

Pourquoi SQLite ?

SQLite a été préféré à PostgreSQL pour sa facilité de déploiement : aucun serveur de base de données n'est à installer ni à configurer. Les données sont stockées dans un fichier local `goals.db` créé automatiquement au premier démarrage. Ce choix est adapté à un projet académique tout en répondant pleinement à l'exigence de persistance des données.

2. Architecture

Modèle de données

La base de données contient une seule table `goals` :

Colonne	Type	Contrainte
id	INTEGER	PRIMARY KEY AUTOINCREMENT
title	TEXT	NOT NULL
status	TEXT	NOT NULL, DEFAULT 'En cours'

Routes de l'API

Méthode	Route	Description	Code retour
GET	/health	Health check	200
GET	/goals	Liste de tous les objectifs	200
GET	/goals/:id	Un objectif par son ID	200 / 404
POST	/goals	Crée un nouvel objectif	201 / 400
PUT	/goals/:id	Modifie un objectif	200 / 404
DELETE	/goals/:id	Supprime un objectif	200 / 404

Règles de validation

- **Champ** `title` **manquant ou vide** lors d'un POST ou PUT → réponse `400` avec message `{ "error": "title obligatoire" }`
- **ID inexistant** → réponse `404` avec message `{ "error": "non trouvé" }`

Schéma de fonctionnement

```
Client HTTP
    |
    v
Express (server.js)
    |
    |-- GET /health         --> { status: 'ok' }
    |-- GET /goals          --> SELECT * FROM goals
    |-- GET /goals/:id      --> SELECT * FROM goals WHERE id = ?
    |-- POST /goals         --> INSERT INTO goals ...
    |-- PUT /goals/:id      --> UPDATE goals SET ... WHERE id = ?
    |-- DELETE /goals/:id   --> DELETE FROM goals WHERE id = ?
    |
    v
SQLite (goals.db)
```

3. Pipeline CI/CD

Structure du pipeline

Le pipeline GitHub Actions est défini dans `.github/workflows/ci.yml` et se déclenche sur chaque **push** et chaque **Pull Request**.

Il est composé de 2 jobs exécutés séquentiellement :

```
■■■■■■■■■■■■■     ■■■■■■■■■■■■■
■   Lint    ■ ■■> ■  Tests   ■
■■■■■■■■■■■■■     ■■■■■■■■■■■■■
```

Job 1 — Lint : Vérifie la qualité et la syntaxe du code source avec ESLint. Si le linter détecte une erreur, le pipeline s'arrête et le job Tests ne s'exécute pas.

Job 2 — Tests : S'exécute uniquement si le Lint a réussi. Lance la suite de tests Jest + Supertest contre `server.js`. Un échec de test bloque le merge de la Pull Request.

Technologies utilisées dans le pipeline

- `actions/checkout@v4` : récupère le code source
- `actions/setup-node@v4` avec Node.js 22 : installe l'environnement
- `npm ci` : installe les dépendances de façon déterministe depuis `package-lock.json`

Résultat

Le pipeline est **vert** sur la branche `feature/crud-api` après correction des problèmes de dépendances (voir section 4).

4. Difficultés rencontrées

Problème 1 — Express non trouvé en CI

Symptôme : Le job Tests échouait avec l'erreur `Cannot find module 'express'` malgré un `npm ci` présent dans le workflow.

Cause : Le fichier `package.json` commité sur le dépôt ne contenait pas `express` dans les `dependencies`. Le package avait été installé localement mais les modifications n'avaient pas été committées.

Solution : Committer `package.json` (avec `express` dans `dependencies`) et le `package-lock.json` mis à jour.

Problème 2 — ESLint v10 — configuration invalide

Symptôme : Le job Lint échouait car ESLint v10 utilise un nouveau format de configuration (`eslint.config.js`) incompatible avec l'ancien format `.eslintrc`.

Solution : Remplacement du fichier de configuration par un `eslint.config.js` au format « flat config » compatible ESLint v10.

Problème 3 — Persistance des données absente

Symptôme : L'API utilisait un tableau JavaScript en mémoire (`let goals = []`). Toutes les données étaient perdues à chaque redémarrage du serveur.

Solution : Intégration de SQLite via `better-sqlite3`. La table `goals` est créée automatiquement au démarrage et les données persistent entre les sessions.

5. Démonstration de l'API

Les commandes suivantes permettent de tester l'API en local après `node server.js` :

Health check

```
curl http://localhost:3000/health
# { "status": "ok" }
```

Créer un objectif

```
curl -X POST http://localhost:3000/goals \
  -H "Content-Type: application/json" \
  -d '{"title": "Apprendre Express"}'
# { "id": 1, "title": "Apprendre Express", "status": "En cours" }
```

Lister les objectifs

```
curl http://localhost:3000/goals
# [{ "id": 1, "title": "Apprendre Express", "status": "En cours" }]
```

Modifier un objectif

```
curl -X PUT http://localhost:3000/goals/1 \
  -H "Content-Type: application/json" \
  -d '{"title": "Apprendre Express", "status": "Terminé"}'
```

Supprimer un objectif

```
curl -X DELETE http://localhost:3000/goals/1
# { "message": "supprimé" }
```

Conclusion

Ce projet a permis de mettre en pratique les concepts vus en cours : création d'une API REST avec Express, persistance des données avec SQLite, écriture de tests automatisés avec Jest et Supertest, configuration d'un linter ESLint, et mise en place d'un pipeline CI/CD avec GitHub Actions. L'utilisation de Git avec une branche feature et une Pull Request mergée reflète un workflow de développement professionnel.

Annexes — Captures d'écran

Capture 1 : Réponse du Health Check (curl en PowerShell)

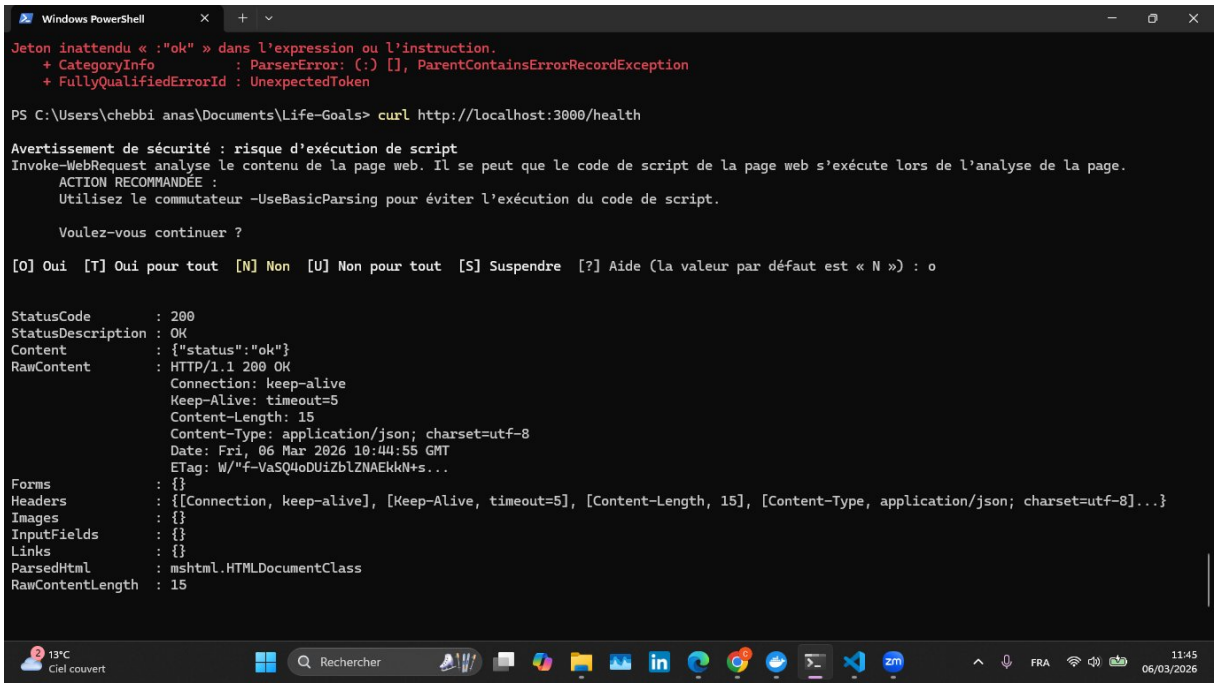

Figure 1 — Résultat de la commande curl sur /health (StatusCode 200, Content: {"status":"ok"})

Capture 2 : Pipeline GitHub Actions

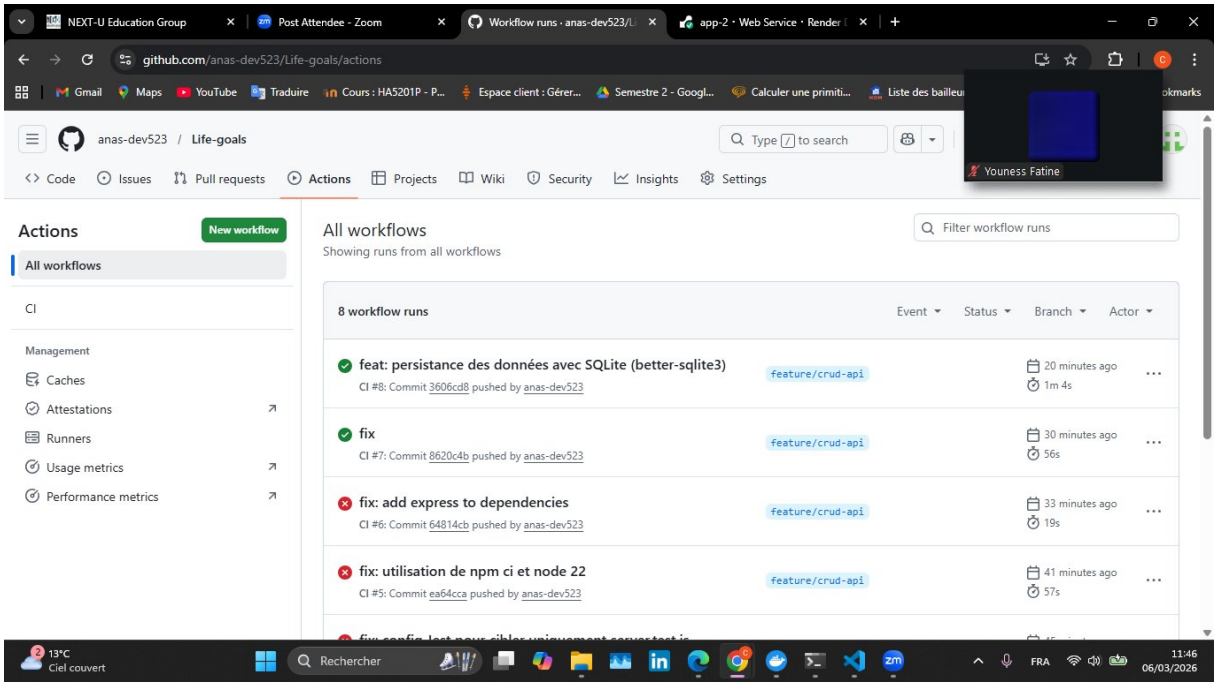

Figure 2 — Historique des workflow runs sur GitHub Actions (pipeline vert après corrections)